

✓ TP IA-GL — Systèmes de recommandation

De la popularité au filtrage collaboratif basé sur les objets

Membres: Yagmur AYDEMIR, Liza BOUROUINA

Formation : Master 2 Informatique — Génie Logiciel, Université de Montpellier

Déclaration d'usage de l'IA : Des outils d'IA générative ont été utilisés à des fins de clarification de concepts, de relecture des formulations et d'aide ponctuelle au débogage. L'implémentation, les choix méthodologiques, l'exécution des expériences, l'analyse des résultats et la validation finale restent notre travail.

✓ Imports et chargement des données

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remc

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# Charger les trois fichiers depuis Google Drive
stars_df = pd.read_csv("/content/drive/MyDrive/user_repo_stars_filt
repos_df = pd.read_csv("/content/drive/MyDrive/github-dataset.csv")
users_df = pd.read_csv("/content/drive/MyDrive/users_stars.csv")

print(f"Taille initiale : {len(stars_df)} interactions, {len(repos_

Taille initiale : 13407 interactions, 1009 repos, 22240 users.
```

✓ Affichage des données

```
print("Repositories : ")
repos_df
```

Repositories :

	repositories	stars_count	forks_count	issues_c
0	octocat/Hello-World	0	0	
1	EddieHubCommunity/support	271	150	
2	ethereum/aleth	0	0	
3	localstack/localstack	0	0	
4	shobhit97/open-gpstracker	0	0	
...	...	...	...	
1004	Tyriar/canvas-astar.dart	2	1	
1005	ankitkumar70777/github-slideshow	0	0	
1006	aitikgupta/interactive_cpu_scheduler	0	5	
1007	gwmccubbin/voting_dapp	11	5	
1008	gamemann/All_PropHealth	4	3	

1009 rows x 7 columns

```
print("Utilisateurs : ")
users_df
```

Utilisateurs :

	login	name	company	location	public_repos
0	schacon	Scott Chacon	@gitbutlerapp	Berlin, Germany	225
1	adelcambre	Andy Delcambre	@stripe	Boulder, CO	100
2	usergenic	Brendan Baldwin	NaN	Eagle, ID	48
3	fdb	Frederik De Bleser	@nodebox	Belgium	21
4	darinel	NaN	NaN	NaN	82
...	...	...	...	...	...
22235	cxml2	CXML2	NaN	NaN	10
22236	Cher172	おじぎ	NaN	コホリント島	10
22237	KrauseFx	Felix Krause	NaN	Vienna, Austria	145
22238	AnkitChouhanDws	NaN	NaN	NaN	7
22239	Novusvetus	Novusvetus / Marcel Rudolf	@ReindeerWeb @FashionRetail	Bückeburg, Germany	1

22240 rows × 14 columns

```
print("Interactions (Repositories/Utilisateurs) : ")
stars_df
```

Interactions (Repositories/Utilisateurs) :

	user	repository
0	ff6347	octocat/Hello-World
1	gmittal	octocat/Hello-World
2	angusshire	octocat/Hello-World
3	Jinxiansen	octocat/Hello-World
4	canering	octocat/Hello-World
...	...	...
13402	Chan9390	defuse/email-spoofing
13403	picatz	defuse/email-spoofing
13404	ur5us	defuse/email-spoofing
13405	MilleniumSpark	marcel-dempers/k8s-build-your-own
13406	MihalczGabor	gamemann/All_PropHealth
13407 rows x 2 columns		

✓ Préparation des données

```
# Supprimer les doublons

stars_df["user"] = stars_df["user"].astype(str).str.strip()
stars_df["repository"] = stars_df["repository"].astype(str).str.strip()
stars_df.drop_duplicates(subset=["user", "repository"], inplace=True)

# Make sure repositories match between files
merged_repos = repos_df[["repositories", "stars_count", "forks_count"]]
merged_repos.rename(columns={"repositories": "repository"}, inplace=True)
merged_repos.drop_duplicates(subset=["repository"], keep="first", inplace=True)

# Keep only repositories that appear in both datasets
stars_df = stars_df[stars_df["repository"].isin(merged_repos["repository"])]

print(f"Taille finale : {len(stars_df)} interactions, {len(merged_repos)} repos")

Taille finale : 13407 interactions, 932 repos, 22240 users.
```

✓ Division TRAIN/TEST (70 / 30)

```
# TRAIN/TEST split : 70/30 par utilisateur

train_list, test_list = [], []

for user, user_df in stars_df.groupby("user"):
    if len(user_df) < 2:
        train_list.append(user_df)
        continue

    train, test = train_test_split(
        user_df,
        test_size=0.3,
        random_state=42
    )
    train_list.append(train)
    test_list.append(test)

train_df = pd.concat(train_list)
test_df = pd.concat(test_list)

print(f"Train: {len(train_df)} interactions, Test: {len(test_df)} interactions")

Train: 9040 interactions, Test: 4367 interactions
```

✓ Étape 1 — Modèle basé sur la popularité

✓ Model basé sur la popularité

```
popular_repos = merged_repos.sort_values(by="stars_count", ascending=False)
print(popular_repos)
```

	repository	stars_count	forks_count
0	iamshaunjp/Complete-React-Tutorial	995	
1	adrianhajdin/project_graphql_blog	977	2
2	adrianhajdin/project_medical_pager_chat	968	3
3	brettkromkamp/contextualise	960	
4	sukritishah15/DS-Algo-Point	956	2
..	...	...	..
927	localstack/localstack	0	
928	rrousselGit/freeze	0	1
929	google/accompanist	0	3
930	WhiteHouse/petitions	0	3
931	mattgallagher/AudioStreamer	0	5

	language
0	NaN
1	JavaScript
2	JavaScript
3	Python
4	Java
..	...
927	Python
928	Dart
929	Kotlin
930	JavaScript
931	Objective-C

[932 rows x 4 columns]

✓ Fonction de recommandation

```
def recommend_popular(user_id, top_k=10):  
    """Return top-k popular repositories that the user hasn't starr  
    # dépôts déjà étoilés par l'utilisateur  
    user_starred = set(train_df.loc[train_df["user"] == user_id, "r  
    # filtrer les dépôts non encore étoilés  
    recs = popular_repos[~popular_repos["repository"].isin(user_sta  
    # top_k  
    return recs.head(top_k)["repository"].tolist()
```

✓ Evaluation

```

# calcul de la précision, recall et f-measure pour le test set
def precision_recall_f1_at_k(train_df, test_df, top_k=10):
    precisions, recalls, f1s = [], [], []

    users = test_df["user"].unique()
    for user in users:
        # test items (ground truth)
        true_items = set(test_df.loc[test_df["user"] == user, "rep")
        if len(true_items) == 0:
            continue

        # recommended items
        recommended = set(recommend_popular(user, top_k=top_k))
        if not recommended:
            continue

        # true positives
        tp = len(true_items & recommended)
        precision = tp / len(recommended) if len(recommended) > 0 e
        recall = tp / len(true_items) if len(true_items) > 0 else 0
        f1 = 2 * precision * recall / (precision + recall) if (prec

        precisions.append(precision)
        recalls.append(recall)
        f1s.append(f1)

    results = {
        "Precision@K": np.mean(precisions),
        "Recall@K": np.mean(recalls),
        "F1@K": np.mean(f1s)
    }
    return results

metrics = precision_recall_f1_at_k(train_df, test_df, top_k=10)

print("\nEvaluation Results (Popularity Model)")
print(f"Precision@10: {metrics['Precision@K']:.4f}")
print(f"Recall@10:      {metrics['Recall@K']:.4f}")
print(f"F1@10:           {metrics['F1@K']:.4f}")

```

```

Evaluation Results (Popularity Model)
Precision@10: 0.0181
Recall@10:    0.0352
F1@10:       0.0229

```

✓ Evaluation pour un user particulier

```
# Sélection d'un utilisateur particulier (par index or manuellement
some_user = stars_df['user'].iloc[40] # ou "Adlinke"
print(f"Evaluating user: {some_user}")

# USER PROFILE
user_profile = users_df[users_df["login"] == some_user]

if not user_profile.empty:
    print("\nUser profile:")
    print(user_profile.to_string(index=False))
else:
    print("No user metadata found")

# Interaction du user
user_train = train_df[train_df["user"] == some_user]
user_test = test_df[test_df["user"] == some_user]

print(f"\nTrain interactions: {len(user_train)} repositories")
print(f"\nTest interactions: {len(user_test)} repositories")

if len(user_train) > 0:
    print("\nTrain repositories:")
    print(user_train["repository"].to_list())

if len(user_test) > 0:
    print("\nTest repositories:")
    print(user_test["repository"].to_list())

# Liste des recommandations
recommendations = recommend_popular(some_user, top_k=10)
print("\nTop-10 Recommended repositories:")
print(recommendations)

# EVALUATION
true_items = set(user_test["repository"])

if len(true_items) == 0:
    print("No test items found")
else:
    recommended = set(recommendations)
    tp = len(true_items & recommended)

    if tp > 0:
        print(f"\nCorrectly recommended repositories: {list(true_it
```

```

precision = tp / len(recommended) if len(recommended) > 0 else
recall = tp / len(true_items) if len(true_items) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precisio

print("\nUser-specific evaluation results:")
print(f"Precision@10: {precision:.4f}")
print(f"Recall@10:      {recall:.4f}")
print(f"F1@10:          {f1:.4f}")

```

Evaluating user: Adlinke

User profile:

login	name	com
Adlinke	Cap'n Donald Trump & AdLinke	AdLinke Media & Trump Organisat

Train interactions: 16 repositories

Test interactions: 7 repositories

Train repositories:

['PragneshRamani/Coursera---Using-Python-to-Access-Web-Data', 'kdelc

Test repositories:

['taniarascia/primitive', 'google/acai', 'EddieHubCommunity/support'

Top-10 Recommended repositories:

['iamshaunjp/Complete-React-Tutorial', 'adrianhajdin/project_graphql

Correctly recommended repositories: ['sukritishah15/DS-Algo-Point',

User-specific evaluation results:

Precision@10: 0.2000

Recall@10: 0.2857

F1@10: 0.2353

```

top20 = popular_repos.head(20)
print(top20[["repository", "stars_count", "language"]].to_string(in
print()
print(top20["language"].value_counts())

```

	repository	stars_count	
	iamshaunjp/Complete-React-Tutorial	995	
	adrianhajdin/project_graphql_blog	977	
	adrianhajdin/project_medical_pager_chat	968	
	brettkromkamp/contextualise	960	
	sukritishah15/DS-Algo-Point	956	
	EddieHubCommunity/awesome-github-profiles	925	
	adrianhajdin/ecommerce_sanity_stripe	907	
	torvalds/uemacs	898	
	taniaraschia/primitive	879	
	antonmedv/countdown	847	
	adrianhajdin/chat_application	829	
	curiously/Deep-Learning-For-Hackers	824	Jupyter
	xoreaxeaxe/sinkhole	823	
	offensive-security/kali-arm-build-scripts	805	
	stong/infosec-resources	799	
	gunnarmorling/awesome-opensource-data-engineering	790	
	antonmedv/finder	775	
	mscoutermarsh/dotfiles	766	
	btholt/complete-intro-to-containers	765	
	donnemartin/viz	759	

language	
JavaScript	5
Python	2
HTML	2
Java	1
C	1
SCSS	1
Go	1
Jupyter Notebook	1
Assembly	1
Vim Script	1

Name: count, dtype: int64

Quels types de dépôts sont favorisés ?

Le modèle favorise les dépôts ayant le plus grand nombre d'étoiles sur GitHub, ce qui fait ressortir quelques profils bien précis :

- **Les technologies web grand public** : JavaScript domine le top 20 (5/20), suivi de Python et HTML (2/20 chacun).
- **Les projets pédagogiques et tutoriels** : par exemple `iamshaunjp/Complete-React-Tutorial` (1er) ou les projets de `adrianhajdin` (blog GraphQL, chat, e-commerce), qui occupent à eux seuls 4 places dans le top 11. Ce sont des projets « grand public », destinés aux débutants.
- **Les listes de ressources** : `awesome-github-profiles`, `infosec-resources`, `awesome-opensource-data-engineering`. Plusieurs n'ont d'ailleurs aucun langage renseigné (`NaN`), car ce sont des collections de liens plutôt que du code.

À l'inverse, les projets spécialisés (bibliothèques de niche, outils pour un domaine précis, langages peu répandus) n'apparaissent pas.

Remarque : la popularité est calculée à partir de `stars_count`, c'est-à-dire les étoiles globales sur GitHub, et non à partir des interactions de nos utilisateurs. Le modèle favorise donc ce qui est populaire sur GitHub en général, pas forcément ce qui est populaire parmi les utilisateurs de notre jeu de données. C'est l'une des raisons de ses faibles performances.

```
# Évaluation de la baseline popularité pour plusieurs valeurs de K
for k in [5, 10, 15, 20]:
    metrics = precision_recall_f1_at_k(train_df, test_df, top_k=k)
    print(f"K={k} -> Precision: {metrics['Precision@K']:.4f}, "
          f"Recall: {metrics['Recall@K']:.4f}, F1: {metrics['F1@K']:.4f}")
```

```
K=5 -> Precision: 0.0155, Recall: 0.0154, F1: 0.0148
K=10 -> Precision: 0.0181, Recall: 0.0352, F1: 0.0229
K=15 -> Precision: 0.0164, Recall: 0.0477, F1: 0.0234
K=20 -> Precision: 0.0173, Recall: 0.0682, F1: 0.0267
```

Évaluation de la baseline pour $K \in \{5, 10, 15, 20\}$

La Precision reste faible et stable (environ 1,5 à 1,8 %) : la liste recommandée est quasi identique pour tous les utilisateurs, sans lien avec leurs goûts individuels. Le Recall progresse avec K (de 1,5 % à 6,8 %) : plus on propose de dépôts, plus on a de chances de retrouver ceux réellement étoilés par l'utilisateur. Le F1 suit la même tendance que le Recall et reste très faible (0.015 à 0.027).

Quels utilisateurs peuvent être pénalisés par cette approche ?

Les utilisateurs pénalisés sont ceux dont les goûts ne correspondent pas au haut du classement global :

- **Utilisateurs aux langages ou domaines rares** → leurs dépôts préférés n'apparaissent jamais dans le top, donc aucune recommandation ne leur correspond.
- **Utilisateurs aux intérêts très diversifiés** → leurs étoiles sont dispersées sur des projets variés, souvent hors du top global. On le vérifie plus loin : le F1@10 de la popularité tombe à 0.014 pour les utilisateurs de plus de 10 langages.
- **Utilisateurs experts** → ils connaissent déjà les projets populaires (souvent des tutoriels pour débutants) : le modèle n'a rien de nouveau à leur proposer.

Comment ce modèle se comporte-t-il pour un utilisateur ayant des goûts très spécifiques ?

Très mal. Le modèle propose la même liste à tout le monde, sans tenir compte des goûts de chacun. Si les dépôts préférés de l'utilisateur ne figurent pas dans le top K (cas typique d'un langage ou d'un domaine peu répandu), aucune recommandation ne lui correspond : sa Precision et son Recall sont nuls.

Augmenter K n'aide presque pas, car les dépôts suivants du classement restent des projets populaires et généralistes, éloignés de ses centres d'intérêt.

C'est précisément ce que cherchent à corriger les approches personnalisées comme le filtrage collaboratif, présenté à l'étape suivante.

✓ Étape 2 — Filtrage collaboratif basé sur les objets

```
# Construire la liste des utilisateurs et des dépôts (train unique)
liste_users = train_df["user"].unique().tolist()
liste_repos = train_df["repository"].unique().tolist()

print(f"Nombre d'utilisateurs : {len(liste_users)}")
print(f"Nombre de dépôts : {len(liste_repos)}")

# Associer chaque utilisateur/dépôt à un numéro
user_to_index = {user: i for i, user in enumerate(liste_users)}
repo_to_index = {repo: i for i, repo in enumerate(liste_repos)}

# Construire la matrice utilisateur-dépôt (1 si étoilé, 0 sinon)
matrice = np.zeros((len(liste_users), len(liste_repos)))

for _, ligne in train_df.iterrows():
    i = user_to_index[ligne["user"]]
    j = repo_to_index[ligne["repository"]]
    matrice[i, j] = 1

print("Taille de la matrice :", matrice.shape)
```

```
Nombre d'utilisateurs : 901
Nombre de dépôts : 577
Taille de la matrice : (901, 577)
```

```
# Calcul de la similarité cosinus entre chaque paire de dépôts.

nb_repos = matrice.shape[1]
similarite_depots = np.zeros((nb_repos, nb_repos))

for i in range(nb_repos):
    vecteur_i = matrice[:, i]    # colonne i = qui a étoilé le dépôt
    for j in range(nb_repos):
        if i == j:
            continue    # un dépôt n'est pas son propre voisin
        vecteur_j = matrice[:, j]

        produit_scalaire = np.dot(vecteur_i, vecteur_j)
        norme_i = np.linalg.norm(vecteur_i)
        norme_j = np.linalg.norm(vecteur_j)

        if norme_i > 0 and norme_j > 0:
            similarite_depots[i, j] = produit_scalaire / (norme_i * norme_j)

print("Taille de la matrice de similarité :", similarite_depots.shape)

Taille de la matrice de similarité : (577, 577)
```

```

def recommend_item_based_N(user_id, matrice_sim_N, top_k=10):
    """Recommande à un utilisateur les dépôts les plus similaires à
    if user_id not in user_to_index:          # utilisateur inconnu
        return []
    i_user = user_to_index[user_id]
    depots_etoiles = matrice[i_user]          # vecteur binaire des
    scores = np.zeros(len(liste_repos))
    for j in range(len(liste_repos)):
        if depots_etoiles[j] == 1:            # on ne recommande pa
            continue
        for k in range(len(liste_repos)):
            if depots_etoiles[k] == 1:
                # score de j = somme de ses similarités avec les dé
                scores[j] += matrice_sim_N[j, k]
    scores[depots_etoiles == 1] = -np.inf      # exclut définitiveme
    ordre = np.argsort(-scores)[:top_k]       # tri décroissant des
    return [liste_repos[idx] for idx in ordre]

def evaluate_avec_N(matrice_sim_N, Ks=(5, 10, 15, 20)):
    res = {K: ([], [], []) for K in Ks}
    for user, truth in test_by_user.items():
        recs = recommend_item_based_N(user, matrice_sim_N, top_k=ma
        for K in Ks:
            tp = len(set(recs[:K]) & truth)
            p = tp / K
            r = tp / len(truth)
            f = 2 * p * r / (p + r) if (p + r) > 0 else 0
            res[K][0].append(p); res[K][1].append(r); res[K][2].app
    return pd.DataFrame(
        {K: [np.mean(x) for x in res[K]] for K in Ks},
        index=["Precision", "Recall", "F1"]
    ).T.rename_axis("K")

```

```

def limiter_voisins(matrice_sim, N):
    nb_depots = matrice_sim.shape[0]
    nouvelle_matrice = np.zeros_like(matrice_sim)
    for i in range(nb_depots):
        ligne = matrice_sim[i]
        # indices des N valeurs les plus grandes de cette ligne
        indices_top_n = np.argsort(-ligne)[:N]
        for j in indices_top_n:
            nouvelle_matrice[i, j] = ligne[j]
    return nouvelle_matrice

```

Tester avec N=3,N=5,N=10,N=20

```
# Dépôts de test de chaque utilisateur (vérité terrain pour l'évalu
test_by_user = test_df.groupby("user")["repository"].apply(set).to_

matrice_limitee_3 = limiter_voisins(similarite_depots, 3)
print("N=3")
print(evalue_avec_N(matrice_limitee_3))
```

```
N=3
      Precision      Recall      F1
K
5      0.203108  0.217977  0.204544
10     0.158935  0.344572  0.212216
15     0.129782  0.420752  0.194077
20     0.107159  0.458196  0.170404
```

```
matrice_limitee_5 = limiter_voisins(similarite_depots, 5)
print("N=5")
print(evalue_avec_N(matrice_limitee_5))
```

```
N=5
      Precision      Recall      F1
K
5      0.206659  0.223914  0.209154
10     0.169478  0.366748  0.225888
15     0.137181  0.445332  0.205115
20     0.115205  0.495345  0.183153
```

```
matrice_limitee_10 = limiter_voisins(similarite_depots, 10)
print("N=10")
print(evalue_avec_N(matrice_limitee_10))
```

```
N=10
      Precision      Recall      F1
K
5      0.221976  0.238747  0.223517
10     0.171587  0.370485  0.228377
15     0.142508  0.461124  0.212784
20     0.117647  0.508318  0.187257
```

```
matrice_limitee_20 = limiter_voisins(similarite_depots, 20)
print("N=20")
print(evalue_avec_N(matrice_limitee_20))
```

N=20			
	Precision	Recall	F1
K			
5	0.225083	0.243390	0.227032
10	0.172919	0.376007	0.230538
15	0.142138	0.461899	0.212295
20	0.121976	0.527592	0.194078

Effet du nombre de voisins N : la performance augmente légèrement et de façon régulière avec N (F1@10 : 0.212 pour N=3, 0.226 pour N=5, 0.228 pour N=10, 0.231 pour N=20), mais les gains s'aplatissent au-delà de N=5. Avec un petit N, chaque dépôt n'a que quelques voisins : beaucoup de dépôts candidats obtiennent un score nul et le classement devient peu informatif. Un N plus grand permet d'accumuler davantage de signaux faibles.

Quelle que soit la valeur de N, l'item-based dépasse très largement la baseline popularité sur toutes les métriques et tous les K (F1 environ 7 à 15 fois supérieur).

✓ Étape 3 — Évaluation et comparaison

```

import ast
rows = []
for u, truth in test_by_user.items():
    n_train = (train_df.user == u).sum()
    langs = users_df.loc[users_df.login == u, "languages_used"]
    n_lang = len(ast.literal_eval(langs.iloc[0])) if len(langs) and
    for name, recs in [("pop", recommend_popular(u, 10)),
                      ("item", recommend_item_based_N(u, matrice_l
    tp = len(set(recs) & truth); p, r = tp/10, tp/len(truth)
    rows.append(dict(user=u, model=name, n_train=n_train, n_lang=n_lang,
                    f1=2*p*r/(p+r) if p+r else 0))

df = pd.DataFrame(rows)
df["act_bin"] = pd.qcut(df.n_train, 4, duplicates="drop")
df["lang_bin"] = pd.cut(df.n_lang, [-1, 2, 5, 10, 100])

print("F1@10 selon le nombre d'étoiles en train :")
print(df.pivot_table(index="act_bin", columns="model", values="f1",
print(df[df.model == "item"].groupby("act_bin", observed=True).size

print("\nF1@10 selon le nombre de langages utilisés :")
print(df.pivot_table(index="lang_bin", columns="model", values="f1",
print(df[df.model == "item"].groupby("lang_bin", observed=True).siz

```

F1@10 selon le nombre d'étoiles en train :

model	item	pop
act_bin		
(6.999, 8.0]	0.203856	0.015389
(8.0, 11.0]	0.248397	0.021850
(11.0, 63.0]	0.271103	0.043556
act_bin		
(6.999, 8.0]	452	
(8.0, 11.0]	271	
(11.0, 63.0]	178	

Name: nb_users, dtype: int64

F1@10 selon le nombre de langages utilisés :

model	item	pop
lang_bin		
(-1, 2]	0.293065	0.028822
(2, 5]	0.237640	0.031858
(5, 10]	0.229077	0.030869
(10, 100]	0.210786	0.014479
lang_bin		
(-1, 2]	131	
(2, 5]	119	
(5, 10]	212	
(10, 100]	437	

Name: nb_users, dtype: int64

Quels utilisateurs bénéficient le plus de chaque approche ?

Popularité : elle avantage les utilisateurs aux goûts mainstream (langages très répandus comme JavaScript, projets déjà connus), dont les dépôts préférés figurent déjà dans le top global. Son F1@10 reste faible dans tous les groupes (~0.015 à 0.044). Il augmente un peu avec l'activité et chute pour les profils très diversifiés (0.014 pour >10 langages), dont les intérêts sortent du top global.

Item-based : il avantage les utilisateurs avec un historique riche et cohérent. Le F1@10 passe de 0.204 (7–8 étoiles en train, 452 utilisateurs) à 0.271 (12 étoiles ou plus, 178 utilisateurs), car plus on a d'étoiles à comparer, plus les scores de similarité sont fiables. Une partie de ce gain est toutefois mécanique : un utilisateur plus actif a aussi plus de dépôts en test, ce qui relève le plafond atteignable de la précision. Il fonctionne aussi mieux pour les utilisateurs spécialisés (0.293 pour ≤ 2 langages, 131 utilisateurs) que pour les généralistes (0.211 pour >10 langages, 437 utilisateurs). Des goûts concentrés produisent des voisinages cohérents, alors que des intérêts dispersés diluent les scores.

Conclusion : l'item-based domine dans tous les segments. Même son pire groupe (0.204) dépasse largement le meilleur groupe de la popularité (0.044). Tous les groupes comptent au moins 119 utilisateurs, ce qui rend ces comparaisons fiables.

```
for name, f in [("pop", lambda u: recommend_popular(u, 10)),
               ("item", lambda u: recommend_item_based_N(u, matric
recs = set(r for u in test_by_user for r in f(u))
print(name, len(recs), f"{len(recs)/len(liste_repos):.1%}")
```

```
pop 15 2.6%
item 428 74.2%
```

Les systèmes recommandent-ils toujours les mêmes dépôts ? Nouvel utilisateur sans interaction ?

Nous avons mesuré la couverture du catalogue, c'est-à-dire la proportion de dépôts distincts recommandés au moins une fois (top-10, sur les 577 dépôts présents en train).

La popularité ne recommande que 15 dépôts distincts (2.6 %) à l'ensemble des utilisateurs. La liste est quasiment identique pour tous. Elle varie à peine, et seulement parce qu'on retire les dépôts déjà étoilés, qui sont alors remplacés par les suivants du classement. 97 % du catalogue n'est jamais proposé.

L'item-based recommande 428 dépôts distincts (74.2 %). Les recommandations dépendent des dépôts déjà étoilés par chacun, ce qui donne une bien plus grande diversité entre utilisateurs. Le système reste cependant limité aux dépôts déjà étoilés par au moins un utilisateur en train.

Nouvel utilisateur sans interaction : pour l'item-based, `depots_etoiles` est vide, donc tous les scores restent à 0 : impossible de recommander quoi que ce soit. C'est le problème classique du cold-start utilisateur. La popularité, elle, continue de fonctionner normalement, car elle ne dépend pas de l'historique individuel.

```
repos_train = set(train_df["repository"])
test_cold = test_df[~test_df["repository"].isin(repos_train)]
print(f"Dépôts présents en test mais absents du train : {len(test_cold)}")
print(f"Interactions de test concernées : {len(test_cold)} / {len(test_df)} ({len(test_cold)/len(test_df):.1%})")
```

```
Dépôts présents en test mais absents du train : 53
Interactions de test concernées : 81 / 4367 (1.9%)
```

Que se passe-t-il lorsqu'un nouveau dépôt vient d'être créé et n'a encore reçu aucune étoile ?

Item-based : la matrice utilisateur-dépôt est construite à partir du train. Un dépôt sans aucune étoile n'y apparaît donc pas du tout, et il ne peut jamais être recommandé. C'est le problème du cold-start objet. On l'observe directement dans nos données : 53 dépôts présents en test sont absents du train. Les 81 interactions de test correspondantes (1.9 % du test) sont donc impossibles à retrouver par l'item-based. L'effet sur le rappel global reste faible ici, mais il serait bien plus marqué sur une plateforme réelle comme GitHub, où de nouveaux dépôts sont créés en permanence.

Popularité : un dépôt à 0 étoile se retrouve tout en bas du classement et n'est jamais proposé non plus. Ce modèle renforce les dépôts déjà populaires au détriment des nouveaux.

Solution possible : le filtrage basé sur le contenu ne dépend pas des interactions. Un nouveau dépôt possède déjà des caractéristiques (langage, forks, etc.), ce qui permet de le rapprocher de dépôts similaires dès sa création.

Pourquoi ce chiffre est faible : un dépôt n'est absent du train que si toutes ses étoiles tombent dans le test (30 % de chances par étoile). Les dépôts étant généralement étoilés par plusieurs utilisateurs, ce cas est rare.

Quels avantages et limites observez-vous ?

Popularité

Avantages :

Simple et rapide à mettre en place, sans aucun calcul de similarité. Fonctionne même pour un utilisateur tout nouveau, sans historique, puisqu'elle ne dépend pas de ses goûts.

Limites :

Performances très faibles : $F1@10 = 0.023$, avec une précision qui stagne autour de 1.5–1.8 % quel que soit K. Aucune personnalisation : seulement 15 dépôts distincts recommandés (2.6 % du catalogue), la liste est quasiment la même pour tous. Biais de popularité : elle renforce les dépôts déjà populaires au détriment de projets plus discrets, et un nouveau dépôt (0 étoile) n'est jamais proposé. Dépend

de la qualité de stars_count : certains dépôts étoilés dans nos données ont stars_count = 0 dans les métadonnées (par exemple octocat/Hello-World), ce qui fausse le classement.

Item-based

Avantages :

Recommandations réellement personnalisées : F1@10 = 0.231 (N=20), soit environ 10 fois mieux que la popularité, et ce pour toutes les valeurs de K et de N. Bien plus de diversité : 428 dépôts distincts recommandés (74.2 % du catalogue). Domine dans tous les profils d'utilisateurs : même son pire groupe (0.204) dépasse largement le meilleur groupe de la popularité (0.044). N'a besoin d'aucune métadonnée sur les dépôts, seulement des interactions.

Limites :

Dépend de l'historique : les performances baissent pour les utilisateurs peu actifs (0.204 contre 0.271 pour les plus actifs) et pour les profils dispersés sur beaucoup de langages (0.211 pour plus de 10 langages). Cold-start : impossible de recommander à un nouvel utilisateur (aucune étoile) ou de recommander un nouveau dépôt (absent de la matrice). Dans nos données, 1.9 % des interactions de test portent sur des dépôts absents du train et sont donc introuvables. Plus coûteux à calculer : la matrice de similarité est de taille (nombre de dépôts)², ce qui devient lourd sur un vrai catalogue comme GitHub.

Bilan : les deux approches sont complémentaires. Une solution hybride utiliserait l'item-based quand l'utilisateur a assez d'historique, et la popularité (ou le filtrage basé sur le contenu) en cas de cold-start.

